

Cola de React — Meus Livros

Oct 7, 2026 · @Jefferson

Como usar a cola

Consulte quando esquecer algo, como um profissional consulta a documentação. Cada conceito tem três partes: **o que faz**, **por que usar** e **um exemplo do projeto Meus Livros**.

Antes de olhar a cola, tente lembrar sozinho por alguns segundos. É esse esforço que faz o conhecimento ficar.

Componente

O que faz: é uma função que devolve um pedaço de tela.

Por que usar: você cria a peça uma vez e usa quantas vezes quiser.

Regras: o nome começa com letra maiúscula, e o `return` devolve um pacote só (se forem vários elementos, ficam dentro de um `<div>`).

```
function Livro() {  
  return <h2>Dom Casmurro</h2>;  
}
```

Para usar: escreva o nome como uma tag: `<Livro />` .

Função comum x componente: todo componente é uma função. Mas só é componente se começar com maiúscula e devolver tela.

export e import

O que faz: `export` deixa um componente disponível para outros arquivos. `import` traz esse componente para o arquivo atual.

Por que usar: no jeito profissional, cada componente fica no seu próprio arquivo.

No `Livro.jsx` (última linha):

```
export default Livro;
```

No `App.jsx` (primeira linha):

```
import Livro from "./Livro";
```

Depois de `import` vem o **nome** do que você quer. Depois de `from` vem o **endereço do arquivo**.

Ferramentas do React também são importadas, entre chaves:

```
import { useState, useEffect } from "react";
```

Props

O que faz: entrega informações de um componente para outro. O `App` (o chefe) entrega, e o `Livro` (o funcionário) recebe.

Por que usar: o mesmo componente pode mostrar conteúdos diferentes. O `App` decide o **que** aparece; o `Livro` decide **como** aparece.

Quem **entrega** (`App.jsx`):

```
<Livro titulo="0 Uruguai" autor="Basílio da Gama" />
```

Quem **recebe** (`Livro.jsx`):

```
function Livro(props) {  
  return <h2>{props.titulo}</h2>;  
}
```

O React monta sozinho o objeto `props = { titulo: "0 Uruguai", autor: "Basílio da Gama" }`. Para abrir uma parte, use `props.titulo` (de qual ficha? de `props`).

Aspas ou chaves? Texto fixo vai entre aspas: `titulo="0 Uruguai"`. Conteúdo de uma caixa vai entre chaves: `{props.titulo}`.

As informações sempre **descem**, do chefe para o funcionário.

map no React

O que faz: passa por uma lista, um item de cada vez, e devolve **uma lista nova, do mesmo tamanho, com cada item transformado**.

Por que usar: em vez de escrever um `<Livro />` para cada livro, uma linha só cria todos. Um livro novo na lista aparece na tela sozinho.

```
{livros.map((livro) => (
  <Livro
    key={livro.titulo}
    titulo={livro.titulo}
    autor={livro.autor}
  />
))}
```

Nome	O que é	Onde existe
livros	a lista inteira	no App inteiro
livro	o item da vez	só dentro do map

O key : o React pede uma identificação única para cada item da lista. Por enquanto, usamos o título.

useState

O que faz: cria um valor que, quando muda, faz o React atualizar a tela sozinho.

Por que usar: se uma variável comum muda, a tela não muda, porque o React não percebe.

```
const [lido, setLido] = useState(false);
```

Parte	O que é
lido	a caixa que guarda o valor
setLido	a caixa com a função que muda o valor
false	o valor inicial

A função set faz sempre duas ações: troca o valor e avisa o React para desenhar a tela de novo.

Regras:

- Quer **mudar** algo? Use a caixa com `set: setLido(true)`. Nunca escreva `lido = true`.
- Quer **saber** algo? Use a caixa sem `set: {lido && <p>Lido</p>}`.
- Os nomes seguem o padrão: `valor` e `setValor` (`livros` e `setLivros`, `novoTitulo` e `setNovoTitulo`).
- O `useState` só funciona **dentro** de um componente.

Valores iniciais comuns: `false` (não/falso), `0` (número), `""` (texto vazio), `[]` ou uma lista (lista).

Eventos: onClick e onChange

O que faz: diz o que acontece quando o usuário interage com a tela.

Por que usar: é assim que um botão ou um campo de texto fazem alguma coisa.

onClick : roda quando alguém clica. No React se escreve com C maiúsculo.

```
<button onClick={() => setLido(true)}>Marcar como lido</button>
<button onClick={adicionarLivro}>Adicionar</button>
```

- Com `() =>` : quando a função precisa receber um valor, como `setLido(true)`.
- Sem parênteses: quando você só **entrega** uma função pronta, como `adicionarLivro`. Com parênteses, ela rodaria ao abrir a página, antes do clique.

onChange : roda a cada letra digitada num campo.

```
<input
  value={novoTitulo}
  onChange={(evento) => setNovoTitulo(evento.target.value)}
/>
```

`evento.target.value` é o texto que está no campo, o mesmo `.value` do Setting.

Adicionar item na lista com ...

O que faz: `[...livros, novoLivro]` cria uma lista nova com todos os livros antigos e, no final, o livro novo.

Por que usar: o React só atualiza a tela quando recebe uma **lista nova** pelo `set`. O `.push()` mexe na lista antiga sem avisar o React.

```
function adicionarLivro() {
  if (novoTitulo === "") {
    return;
  }
  const novoLivro = { titulo: novoTitulo };
  setLivros([...livros, novoLivro]);
  setNovoTitulo("");
}
```

1. Se o campo estiver vazio, o `return` para a função ali.
2. Monta o livro novo com o texto digitado.
3. Coloca o livro novo na lista.
4. Limpa o campo.

useEffect

O que faz: roda um código sempre que uma caixa vigiada muda.

Por que usar: para fazer algo automaticamente quando um dado muda, como salvar no cofre. No Setting, você precisava lembrar de chamar `salvarPacientes()` em cada lugar.

```
useEffect(() => {
  localStorage.setItem("livros", JSON.stringify(livros));
}, [livros]);
```

- A primeira parte é o código que vai rodar.
- `[livros]` é a caixa vigiada: toda vez que ela mudar, o código roda de novo.
- Precisa ser importado: `import { useState, useEffect } from "react";`

Cofre: localStorage, stringify, parse e ||

O que faz: guarda dados no navegador, para não sumirem ao recarregar a página.

Regra principal: o localStorage **só aceita texto**.

Função	O que faz	Quando usar
<code>JSON.stringify</code>	lista em texto	para guardar no cofre
<code>JSON.parse</code>	texto em lista	para tirar do cofre

Função	O que faz	Quando usar
<code>localStorage.setItem</code>	guarda numa gaveta	junto com o <code>stringify</code>
<code>localStorage.getItem</code>	pega de uma gaveta (<code>null</code> se vazia)	junto com o <code>parse</code>

Para lembrar: **stringify** vem de *string*, que quer dizer texto. Ele **vira texto**.

Ler o cofre ao abrir a página:

```
const livrosSalvos = JSON.parse(localStorage.getItem("livros"));
const [livros, setLivros] = useState(livrosSalvos || [ /* lista inicial */
]);
```

O `||` quer dizer **ou**: "use os livros salvos ou, se o cofre estiver vazio (`null`), a lista inicial".

Escopo e ordem de leitura

Escopo é o lugar onde uma variável existe. Antes de usar uma caixa, pergunte: **onde ela foi criada?**

Caixa	Criada em	Existe em
<code>livros</code>	topo do <code>App</code>	no <code>App</code> inteiro
<code>novoLivro</code>	dentro de <code>adicionarLivro</code>	só dentro dessa função
<code>livro</code>	pelo <code>map</code>	só dentro do <code>map</code>

Ordem de leitura: o JavaScript lê de cima para baixo. Uma caixa só pode ser usada **depois** da linha que a cria. Por isso o `livrosSalvos` fica acima do `useState` dos livros.

Também não dá para usar uma caixa para criar ela mesma: `const novoLivro = { titulo: novoLivro }` dá erro.

Meus erros recorrentes

Confira esta lista antes de mandar o código.

Erro	Exemplo	Pergunta para evitar
Usar a gaveta sem a ficha	<code>titulo</code> em vez de <code>props.titulo</code>	De qual ficha?
Singular e plural	<code>livro</code> em vez de <code>livros</code>	É a lista inteira ou o item da vez?
Letras trocadas	<code>livro.anp</code> em vez de <code>livro.ano</code>	O nome está idêntico ao de onde foi criado?
Copiar nome de outro exemplo	<code>nota</code> para guardar a lista de livros	O nome diz o que está dentro?
Trocar valor e função	<code>lido(true)</code> em vez de <code>setLido(true)</code>	Quero mudar (com <code>set</code>) ou saber (sem <code>set</code>)?
Esquecer parênteses	<code>setLido</code> em vez de <code>setLido(false)</code>	A função está sendo chamada com o valor?
Esquecer a vírgula na lista	<code>}</code> em vez de <code>},</code>	Cada item está separado do próximo?
Usar caixa antes de criar	<code>livrosSalvos</code> usado acima da sua linha	A caixa foi criada mais acima?

Erros fora do código: se `npm run dev` der erro de `package.json`, use `ls` para ver se está na pasta certa. Se a tela não mudar, confira se o arquivo foi salvo e se é o arquivo do projeto certo.